

→ Bottom-up parsing is

Main idea:

Top-down: Starting from start symbol and try to generate the input.

Bottom up: Start from the input and try to reach the start symbol.

→ What is Bottom-UP parsing?

Bottom-UP parsing is a parsing technique in which we start with the **input string** and gradually reduce it to the **start symbol** of the grammar.

TOP-Down

Start symbol



Grammar



Input

Bottom-UP

Input



Reduce



Start Symbol

bottom up parsing:

input → Start Symbol

Eg; Consider:

$$E \rightarrow E + T$$

$$E \rightarrow T$$

$$T \rightarrow id$$

Suppose input is:

id + id

⇒ We start with the input:

id + id

Now identify piece that can be reduced.

Since:

$$T \rightarrow id$$

we can replace each id by T.

id + id

↓

T + id

↓

T + T

Now :

$$E \rightarrow E + T$$

$$E \rightarrow T$$

We can reduce :

~~Reduce~~ ~~Reduce~~ $E \rightarrow T$

$$T + T$$

↓

$$E + T$$

↓

$$E$$

So :

$$id + id$$

↓

$$T + T$$

↓

$$E + T$$

↓

$$E$$

∴ String is accepted.

→ Bottom-Up parsing uses Reverse Rightmost derivation.

• Top-down parsing:

Uses:

Leftmost derivation

Start symbol $\rightarrow$ Input

• Bottom-up parsing:

Uses:

Reverse of rightmost derivation

Input $\rightarrow$ Start symbol

Eg: rightmost derivation:

E

↓

E + T

↓

E + id

↓

T + id

↓

id + id

Bottom-up parsing reverses these steps:

$id + id$

↓

$T + id$

↓

$E + id$

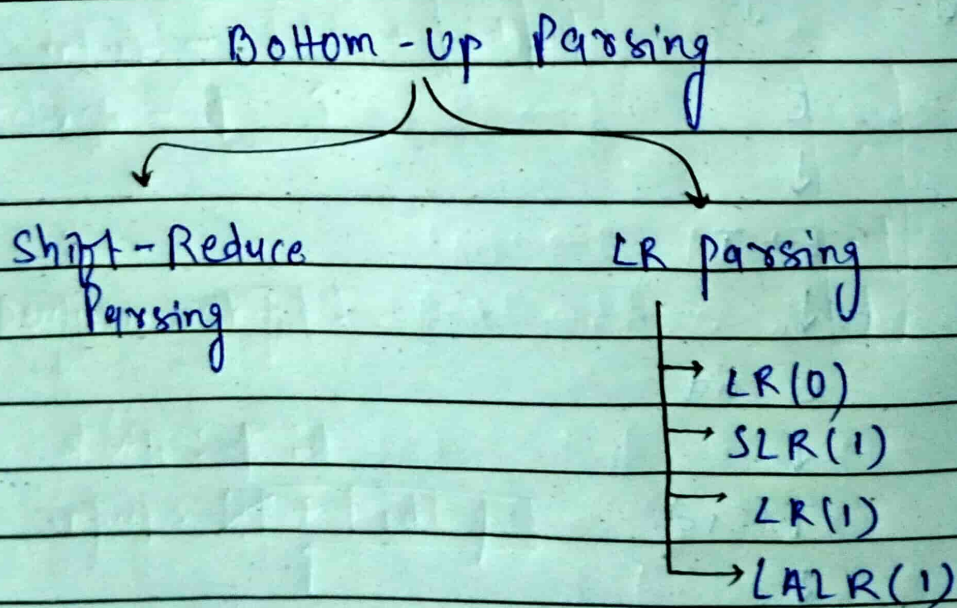
↓

$E + T$

↓

E

→ Types of Bottom-up parsing:



→ Shift-Reduce Parsing:

Shift-Reduce Parsing is the basic technique used in bottom-up parsing.

main idea:

Start with the input string, put symbols into a stack, and repeatedly shift and reduce until we reach the start symbol.

In top-down parsing

Start symbol
↓
Input

In shift-reduce parsing:

Input
↓
Stack
↓
Reduce
↓
Start symbol

→ The four main Actions:

A shift-reduce parser has four possible actions:

1. Shift
2. Reduce
3. Accept
4. Error

(i) Shift:

means:

Take the next symbol from the input and push it onto the stack.

eg:

input →

id	+	id	\$
----	---	----	----

Stack - \$ ↑

perform shift:

Stack: \$ id

input:

id	+	id	\$
---------------	---	----	----

 ↑

id
\$

(ii) Reduce :

means :

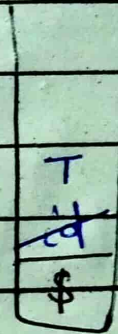
Find a sequence on the top of the stack that matches the RHS of a grammar production and replace it with the LHS non-terminal.

Eg ; Grammar :

$T \rightarrow id$

Stack $\rightarrow \$ id$

we can reduce :



$id \rightarrow T$

So : before :

$\$ id$

after :

$\$ T$

This sequence id is called a handle.
{ we study later }

(iii) Accept:

When the complete input has been processed and the Stack contains the Start symbol, the parser accepts.

Eg;

Stack: \$ E

Input: \$

∴

Accept.

E
\$

iv, Error:

If the parser cannot perform a valid shift or reduction, the input is rejected.

Eg: for shift Reducing Parsing:

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow id$

input:

id+id

$\Rightarrow$

input:

id	+	id	\$
----	---	----	----

↑

Step 1: shift id

input:

+	id	\$
---	----	----

↑

T
id
\$

Step 2: Reduce

$T \rightarrow id$

reduce:

$id \rightarrow T$

+
T
\$

Step 3: shift +

id	\$
---------------	----

Step 4: shift id

input:

\$

id
+
T
\$

Step 5: Reduce

$T \rightarrow id$

So,

Step 6: Reduce

$E \rightarrow T$

T
+
T
\$

∴

E
+
T
\$

Step 7: Reduce:

$E \rightarrow E + T$

∴

E
\$

Step 8:

Accept:

Stack: $\$ \epsilon$

input - $\$$

Accept.

→ Shift-Reduce Conflicts:

One important problem in shift-reduce is a shift-reduce conflict.

It happens when the parser cannot decide

Should I shift?

or

Should I reduce?

Eg:

input $\rightarrow + id \$$

Stack $\rightarrow \$ \epsilon$

Depending on the grammar, the parser may multiple possible actions.

This creates:

Shift-Reduce Conflicts.

→ Reduce-Reduce Conflict:

Another problem is :

Reduce using production 1
or

Reduce using production 2

This is called Reduce-Reduce conflict.

Eg;

$A \rightarrow \alpha$

$B \rightarrow \alpha$

If the parser sees α , it may not know whether to reduce:

$\alpha \rightarrow A$

or

$\alpha \rightarrow B$

This creates:

Reduce-Reduce conflict

→ Handle :

A handle is a substring of the current sentential form that matches the RHS (Right-hand side) of a production and whose reduction to the LHS (left-hand side) represents one step of the reverse rightmost derivation.

In simple words :

Handle → the part that we should reduce next.

Eg :

Consider the grammar :

$$\begin{aligned} E &\rightarrow E + T \\ E &\rightarrow T \\ T &\rightarrow id \end{aligned}$$

input : $id + id$

=> During bottom-up parsing, suppose we have :

$id + id$

The first id ~~can~~ can be reduced using :

$$T \rightarrow id$$

$\therefore id + id$

↑

handle

reduces:

$id + id$

↓

$T + id$

so id was the handle

→ Handle pruning:

The repeated process of finding and reducing handle is called

Handle Pruning

Find handle

↓

Reduce handle

↓

Find next handle

↓

Reduce

↓

...

↓

Start Symbol

Done